

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri
Veri Temizleme ve ETL Projesi

Enes Yıldız

22290180

github.com/enesoglu/blm4522

10.06.2026

İÇİNDEKİLER

1. Giriş	3
2. ETL Mimarisi	4
3. Veri Çıkarma (Extract)	6
4. Veri Kalitesi Profillemesi	8
5. Veri Temizleme (Cleaning)	11
6. Veri Dönüştürme (Transform)	13
7. Veri Yükleme (Load)	15
8. Veri Kalitesi Raporları	17
9. Bulgular ve Sonuçlar	19

1. GİRİŞ

1.1 Proje Amacı

Bu teknik rapor, PostgreSQL 18 üzerinde saf SQL temelli bir veri temizleme ve ETL süreci tasarımını akademik ve uygulanabilir biçimde açıklamaktadır. ETL; verinin kaynaktan çıkarılması (extract), hedef sisteme uygun biçime dönüştürülmesi (transform) ve güvenilir modele yüklenmesi (load) aşamalarından oluşan uçtan uca veri mühendisliği yaklaşımıdır. Projenin kaynak verisi, müşteri ve satış alanlarını içeren, kasıtlı olarak kirlenilmiş yaklaşık 5000 satırlık bir CSV dosyasıdır.

Veri kalitesi; doğruluk, tutarlılık, tekillik, bütünlük, zaman geçerliliği ve biçim standardı gibi ölçütlerin birlikte yönetilmesini gerektirir. Bu projede kirli veri türleri yalnızca tespit edilmemiş, aynı zamanda izlenebilir SQL adımlarıyla temizlenmiş, reddedilen kayıtlar ayrı bir tabloya ayrılmış ve nihai kalite raporları ölçülebilir metrikler üzerinden üretilmiştir.

- Kaynak CSV dosyasının staging tablosuna ham biçimiyle alınması.
- NULL, duplicate, format hatası ve aykırı değer profillemesinin yapılması.
- Boşluk, büyük-küçük harf, telefon, e-posta, tarih ve sayısal alanların temizlenmesi.
- Üretilmiş kolonlar, lookup eşleşmeleri ve CTE tabanlı dönüşümlerin uygulanması.
- Duplicate eleme, final tabloya yükleme ve reddedilen kayıtların ayrıştırılması.
- Öncesi/sonrası veri kalitesi raporlarının sayısal olarak üretilmesi.

1.2 Metodoloji

Metodoloji, Extract -> Profile -> Clean -> Transform -> Load -> Report döngüsü üzerine kurulmuştur. Bu döngüde her aşama ayrı SQL betikleriyle yürütülür; her betik çalıştırdıktan sonra satır sayısı, hata sayısı ve kalite metrikleri quality_log tablosuna kaydedilir. Böylece ETL süreci yalnızca veri aktarımı değil, ölçülebilir ve tekrarlanabilir bir veri kalite hattı olarak ele alınır.

Tablo 1. ETL metodoloji döngüsü

Aşama	Amaç	Temel çıktı
Extract	CSV dosyasını ham veri olarak almak	staging_customers
Profile	Kirli veri türlerini ölçmek	Profil metrikleri
Clean	Standart dışı değerleri düzeltmek	cleaned CTE çıktısı
Transform	İş kurallarını ve üretilmiş kolonları eklemek	Dönüştürülmüş aday kayıt
Load	Final tabloya güvenli yükleme yapmak	final_customers
Report	Kalite sonucunu raporlamak	quality_log ve özet tablolar

2. ETL MİMARİSİ

2.1 Genel Veri Akış Diyagramı

Genel veri akışı, ham verinin doğrudan final modele yazılmadığı, bunun yerine izlenebilir ara katmanlardan geçirildiği katmanlı bir mimari olarak tasarlanmıştır. Mimari akış metin olarak aşağıdaki biçimdedir:

```
Kaynak CSV -> Staging -> Cleaning -> Final -> Quality Report
```

Bu akışta staging katmanı veri kaybını önler, cleaning katmanı veri tiplerini ve formatları standartlaştırır, final katman ise yalnızca kontrol edilmiş kayıtları saklar. Quality Report katmanı hem operasyonel izleme hem de akademik değerlendirme için sayısal kanıt üretir.

2.2 Staging Tablo Yaklaşımı

Ham verinin ayrı bir staging tablosuna alınması, ETL tasarımının en kritik kararlarından biridir. Kaynak CSV içindeki hatalar, eksik alanlar ve biçim tutarsızlıkları doğrudan final şemaya zorlanırsa yükleme anında veri kaybı, tip dönüşüm hatası veya sessiz bilgi bozulması oluşabilir. Staging tablosunda tüm alanların text olarak tutulması, hatalı kayıtların analiz edilmesine ve gerektiğinde temizleme kurallarının tekrar çalıştırılmasına imkan verir.

Bu yaklaşım aynı zamanda veri soyağacı (data lineage) ve denetlenebilirlik açısından önemlidir. Her final kaydının hangi staging satırından üretildiği source_staging_id alanı ile izlenebilir; reddedilen satırlar ise ham JSON biçimiyle rejected_records tablosunda korunur.

2.3 Tablo Şemaları

```
-- Ham veriyi kayıpsız almak için staging tablosu
CREATE TABLE staging_customers (
  staging_id      bigserial PRIMARY KEY,
  source_file     text NOT NULL,
  row_number      integer,
  first_name_raw  text,
  last_name_raw   text,
  email_raw       text,
  phone_raw       text,
  age_raw         text,
  city_raw        text,
  country_raw     text,
  order_date_raw  text,
  product_id_raw  text,
  quantity_raw    text,
  unit_price_raw  text,
  loaded_at       timestampz DEFAULT now()
);

-- Temizlenmiş ve iş kuralları uygulanmış final tablo
CREATE TABLE final_customers (
  customer_id      bigserial PRIMARY KEY,
  source_staging_id bigint NOT NULL UNIQUE,
  full_name        text NOT NULL,
  email            text NOT NULL UNIQUE,
  phone            text NOT NULL,
  age              integer CHECK (age BETWEEN 0 AND 120),
  age_group        text NOT NULL,
  customer_segment text NOT NULL,
  city             text NOT NULL,
  country_code     char(2) NOT NULL,
  order_date       date NOT NULL,
  product_id       integer NOT NULL,
  quantity         integer NOT NULL CHECK (quantity > 0),
```

```
unit_price    numeric(12,2) NOT NULL CHECK (unit_price >= 0),
total_amount  numeric(12,2)
              GENERATED ALWAYS AS (quantity * unit_price) STORED,
loaded_at     timestamptz DEFAULT now(),
CHECK (email ~* '^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$'),
CHECK (phone ~ '^\+90[0-9]{10}$')
);

-- Profil ve raporlama metriklerini saklamak için kalite günlüğü
CREATE TABLE quality_log (
  log_id      bigserial PRIMARY KEY,
  run_name    text NOT NULL,
  stage       text NOT NULL,
  metric_name text NOT NULL,
  metric_value numeric(18,4) NOT NULL,
  metric_unit text DEFAULT 'count',
  details     jsonb,
  measured_at timestamptz DEFAULT now()
);
```

3. VERİ ÇIKARMA (EXTRACT)

3.1 Kaynak Dosyanın Yapısı

Kaynak dosya virgülle ayrılmış CSV biçimindedir. Dosyada müşteri adı, iletişim bilgileri, yaş, şehir, ülke, sipariş tarihi, ürün ve tutar alanları yer almaktadır. Aşağıdaki örnek satırlar, veri kümesine kasıtlı olarak eklenen NULL, duplicate, hatalı e-posta, hatalı telefon, tarih uyumsuzluğu, negatif fiyat ve aykırı yaş değerlerini göstermektedir.

```
first_name,last_name,email,phone,age,city,country,order_date,product_id,quantity,unit_price
Enes , Yıldız ,ENES@example.COM,"(532) 111-22-33",24,Ankara,Turkey,2026-05-01,101,2,149.90
Ayşe,Kara,ayse.kara[at]mail.com,0532-999-88-77,31,İstanbul,TR,01/05/2026,102,1,89.50
Mehmet,Demir,mehmet.demir@mail.com,+90 555 333 2211,156,İzmir,Turkey,2026/31/05,103,3,45.00
Zeynep,Ak,zeynep.ak@mail.com,,29,,Turkey,2026-05-04,104,1,-12.00
Enes,Yıldız,enes@example.com,5321112233,24,ANKARA,TR,2026-05-01,101,2,149.90
NULL,Çelik,celik@mail.com,abc-phone,40,Bursa,Turkey,2026-05-07,105,2,120.00
Ali,Veli,,+90 530 111 0000,-4,Konya,TR,05.05.2026,106,1,60.00
```

3.2 COPY Komutu ile Yükleme

```
-- PostgreSQL 18 üzerinde CSV dosyasını staging tablosuna yükleme
COPY staging_customers (
  first_name_raw, last_name_raw, email_raw, phone_raw, age_raw,
  city_raw, country_raw, order_date_raw, product_id_raw,
  quantity_raw, unit_price_raw
)
FROM 'C:/data/dirty_customers.csv'
WITH (
  FORMAT csv,
  HEADER true,
  NULL "",
  ENCODING 'UTF8'
);
```

3.3 Yükleme Sonrası Satır Sayısı Kontrolü

```
-- Yükleme sonrası satır sayısı kontrolü
SELECT COUNT(*) AS staging_row_count
FROM staging_customers;
```

Tablo 2. Extract sonrası satır sayısı

Kontrol	Sonuç
Kaynak CSV beklenen satır sayısı	5000
staging_customers satır sayısı	5000
Eksik veya fazla yüklenen satır	0

4. VERİ KALİTESİ PROFİLLEMESİ

4.1 NULL Analizi

Profil aşamasında ilk kontrol, her sütun için NULL veya boş string oranının ölçülmesidir. Boş string değerleri NULL kabul edilerek hesaplama yapılmıştır; çünkü CSV dosyalarında eksik değerler çoğu zaman ardışık ayrıçlar veya yalnızca boşluk içeren alanlarla temsil edilir.

```
-- Her sütun için NULL/boş değer analizi
SELECT 'first_name' AS column_name,
       COUNT(*) FILTER (WHERE NULLIF(trim(first_name_raw), '') IS NULL) AS null_count
FROM staging_customers
UNION ALL
SELECT 'last_name',
       COUNT(*) FILTER (WHERE NULLIF(trim(last_name_raw), '') IS NULL)
FROM staging_customers
UNION ALL
SELECT 'email',
       COUNT(*) FILTER (WHERE NULLIF(trim(email_raw), '') IS NULL)
FROM staging_customers
UNION ALL
SELECT 'phone',
       COUNT(*) FILTER (WHERE NULLIF(trim(phone_raw), '') IS NULL)
FROM staging_customers
UNION ALL
SELECT 'city',
       COUNT(*) FILTER (WHERE NULLIF(trim(city_raw), '') IS NULL)
FROM staging_customers
UNION ALL
SELECT 'order_date',
       COUNT(*) FILTER (WHERE NULLIF(trim(order_date_raw), '') IS NULL)
FROM staging_customers;
```

Tablo 3. NULL analizi örnek sonucu

Sütun	NULL/boş sayısı	Oran
first_name	112	%2,24
last_name	79	%1,58
email	164	%3,28
phone	221	%4,42
city	268	%5,36

Sütun	NULL/boş sayısı	Oran
order_date	58	%1,16

4.2 Duplicate Analizi

Tekrarlı kayıt analizi e-posta ve normalize edilmiş telefon alanları üzerinden yapılmıştır. Bu iki alan müşteriye tanımlamada yüksek ayırt ediciliğe sahip olduğu için duplicate eleme aşamasında aynı partition anahtarı olarak kullanılmıştır.

```
-- E-posta ve telefon bazında duplicate grupları
WITH normalized AS (
  SELECT
    staging_id,
    lower(trim(email_raw)) AS email_norm,
    regexp_replace(coalesce(phone_raw, ''), '\D', '', 'g') AS phone_digits
  FROM staging_customers
)
SELECT email_norm, phone_digits, COUNT(*) AS duplicate_count
FROM normalized
WHERE email_norm IS NOT NULL AND phone_digits <> ''
GROUP BY email_norm, phone_digits
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC, email_norm
LIMIT 10;
```

Tablo 4. Duplicate analizi örnek sonucu

E-posta	Telefon rakamları	Tekrar sayısı
enes@example.com	5321112233	2
ayse.kara@mail.com	5329998877	3
selin.oz@mail.com	5551203000	2
murat.can@mail.com	905441112233	4

4.3 Format Hataları

Format hataları e-posta, telefon ve tarih alanlarında yoğunlaşmaktadır. E-posta için düzenli ifade (regular expression), telefon için rakamlaştırma sonrası Türkiye GSM standardı, tarih için ise birden çok formatı ayırt eden CASE yapısı kullanılmıştır.

```
-- E-posta, telefon ve tarih format hatalarını sayma
WITH p AS (
  SELECT
    lower(trim(email_raw)) AS email_norm,
    regexp_replace(coalesce(phone_raw, ''), '\D', '', 'g') AS phone_digits,
    trim(order_date_raw) AS date_raw
  FROM staging_customers
)
SELECT
  COUNT(*) FILTER (
    WHERE email_norm !~* '^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$'
  ) AS invalid_email_count,
  COUNT(*) FILTER (
```

```

WHERE phone_digits !~ '^ (90)? [0-9] {10,11} $'
) AS invalid_phone_count,
COUNT(*) FILTER (
  WHERE date_raw !~ '^ \d {4} - \d {2} - \d {2} $'
  AND date_raw !~ '^ \d {2} / \d {2} / \d {4} $'
  AND date_raw !~ '^ \d {2} \. \d {2} \. \d {4} $'
) AS inconsistent_date_count
FROM p;

```

Tablo 5. Format hataları örnek sonucu

Hata tipi	Kayıt sayısı	Açıklama
Geçersiz e-posta	187	Eksik @, hatalı domain veya boş değer
Geçersiz telefon	306	Eksik rakam, metin karakteri veya standart dışı uzunluk
Tutarsız tarih	512	YYYY-MM-DD dışı fakat dönüştürülebilir biçimler dahil

4.4 Aykırı Değerler

```

-- Yaş ve fiyat alanlarında aykırı değer analizi
SELECT
  COUNT(*) FILTER (
    WHERE age_raw !~ '^ -? \d + $'
    OR age_raw::integer < 0
    OR age_raw::integer > 120
  ) AS invalid_age_count,
  COUNT(*) FILTER (
    WHERE unit_price_raw !~ '^ -? \d + ( \. \d + ) ? $'
    OR unit_price_raw::numeric < 0
  ) AS invalid_price_count,
  COUNT(*) FILTER (
    WHERE quantity_raw !~ '^ \d + $'
    OR quantity_raw::integer <= 0
  ) AS invalid_quantity_count
FROM staging_customers;

```

Tablo 6. Profillemeye özet tablosu

Kalite problemi	Tespit edilen kayıt	İşlem kararı
NULL/boş kritik alan	430	Eksik kimlik veya iletişim alanları reddedilir
Duplicate müşteri	248	Window function ile tekilleştirilir
Geçersiz e-posta	187	Düzeltilemeyen kayıt reddedilir
Geçersiz telefon	306	Normalize edilemeyen kayıt reddedilir
Tutarsız tarih	512	Tanımlı formatlardan date tipine çevrilir
Aykırı yaş	93	0-120 aralığı dışı kayıt reddedilir

Kalite problemi	Tespit edilen kayıt	İşlem kararı
Negatif fiyat	74	Satış değeri geçersiz sayılır

5. VERİ TEMİZLEME (CLEANING)

5.1 Boşluk ve Büyük-Küçük Harf Normalleştirme

Temizleme aşamasında metinsel alanlar önce TRIM ile baştaki ve sondaki boşluklardan arındırılmış, e-posta alanı LOWER ile küçük harfe çevrilmiş, ad-soyad ve şehir alanlarında INITCAP kullanılarak raporlama açısından daha okunabilir bir standart elde edilmiştir.

```
-- Metinsel alanlarda boşluk ve harf normalizasyonu
WITH normalized_text AS (
  SELECT
    staging_id,
    initcap(lower(trim(first_name_raw))) AS first_name_clean,
    initcap(lower(trim(last_name_raw))) AS last_name_clean,
    lower(trim(email_raw)) AS email_clean,
    initcap(lower(trim(city_raw))) AS city_clean,
    upper(trim(country_raw)) AS country_clean
  FROM staging_customers
)
SELECT COUNT(*) AS normalized_row_count
FROM normalized_text;
```

Tablo 7. Temizleme adımları öncesi/sonrası

Temizleme adımı	Öncesi sorunlu satır	Sonrası sorunlu satır
Baş/son boşluk	1380	0
E-posta büyük harf kullanımı	914	0
Şehir adı büyük-küçük harf tutarsızlığı	642	0
Boş string değerler	901	0
Telefon özel karakterleri	1734	0

5.2 NULL Yönetimi

NULL yönetiminde iki farklı strateji uygulanmıştır. Kritik alanlarda, örneğin e-posta veya telefon bilgisi düzeltilemiyorsa kayıt final tabloya alınmamıştır. Kritik olmayan şehir alanında ise COALESCE ile 'Bilinmiyor' değeri atanmış ve veri kaybı yerine kontrollü belirsizlik tercih edilmiştir.

```
-- Boş string değerlerini NULL'a çevirme ve varsayılan atama
WITH null_handled AS (
  SELECT
    staging_id,
    NULLIF(trim(first_name_raw), '') AS first_name_nn,
    NULLIF(trim(last_name_raw), '') AS last_name_nn,
    NULLIF(trim(email_raw), '') AS email_nn,
```

```

        NULLIF(trim(phone_raw), "") AS phone_nn,
        COALESCE(NULLIF(trim(city_raw), ""), 'Bilinmiyor') AS city_nn
    FROM staging_customers
)
SELECT
    COUNT(*) FILTER (WHERE email_nn IS NULL) AS missing_email,
    COUNT(*) FILTER (WHERE phone_nn IS NULL) AS missing_phone,
    COUNT(*) FILTER (WHERE city_nn = 'Bilinmiyor') AS default_city
FROM null_handled;

```

5.3 Format Düzeltme

```

-- Telefon standardizasyonu, e-posta validasyonu ve tarih casting
WITH standardized AS (
    SELECT
        staging_id,
        lower(trim(email_raw)) AS email_clean,
        regexp_replace(coalesce(phone_raw, ''), '\D', '', 'g') AS phone_digits,
        trim(order_date_raw) AS date_raw
    FROM staging_customers
),
typed AS (
    SELECT
        staging_id,
        CASE
            WHEN email_clean ~ '*' '[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$'
            THEN email_clean
        END AS email_valid,
        CASE
            WHEN phone_digits ~ '^90[0-9]{10}$' THEN '+' || phone_digits
            WHEN phone_digits ~ '^0[0-9]{10}$' THEN '+90' || substring(phone_digits FROM 2)
            WHEN phone_digits ~ '^[0-9]{10}$' THEN '+90' || phone_digits
        END AS phone_valid,
        CASE
            WHEN date_raw ~ '^[\d{4}-\d{2}-\d{2}]$' THEN to_date(date_raw, 'YYYY-MM-DD')
            WHEN date_raw ~ '^[\d{2}/\d{2}/\d{4}]$' THEN to_date(date_raw, 'DD/MM/YYYY')
            WHEN date_raw ~ '^[\d{2}\.\d{2}\.\d{4}]$' THEN to_date(date_raw, 'DD.MM.YYYY')
        END AS order_date_valid
    FROM standardized
)
SELECT COUNT(*) AS typed_row_count
FROM typed
WHERE email_valid IS NOT NULL
    AND phone_valid IS NOT NULL
    AND order_date_valid IS NOT NULL;

```

5.4 Aykırı Değer Temizleme

```

-- Aykırı yaş, negatif fiyat ve geçersiz miktar temizliği
WITH numeric_clean AS (
    SELECT
        staging_id,
        CASE
            WHEN age_raw ~ '^\d+$' AND age_raw::integer BETWEEN 0 AND 120
            THEN age_raw::integer
        END AS age_clean,
        CASE
            WHEN quantity_raw ~ '^\d+$' AND quantity_raw::integer > 0
            THEN quantity_raw::integer
        END AS quantity_clean,
        CASE
            WHEN unit_price_raw ~ '^\d+(\.\d+)?$'
            AND unit_price_raw::numeric >= 0
            THEN unit_price_raw::numeric(12,2)
        END AS unit_price_clean
    FROM staging_customers
)
SELECT

```

```
COUNT(*) FILTER (WHERE age_clean IS NULL) AS rejected_age,  
COUNT(*) FILTER (WHERE quantity_clean IS NULL) AS rejected_quantity,  
COUNT(*) FILTER (WHERE unit_price_clean IS NULL) AS rejected_price  
FROM numeric_clean;
```

Tablo 8. Aykırı değer temizleme sonucu

Kontrol	Öncesi	Sonrası	Karar
Yaş < 0 veya > 120	93	0	Reddedildi
Negatif fiyat	74	0	Reddedildi
Miktar <= 0	41	0	Reddedildi
Tip dönüşümü yapılamayan sayı	33	0	Reddedildi

6. VERİ DÖNÜŞTÜRME (TRANSFORM)

6.1 Türetilmiş Kolonlar

Dönüşüm aşamasında final modelin analitik sorgulara daha uygun hale gelmesi için full_name, customer_segment ve age_group kolonları türetilmiştir. Segment bilgisi sipariş toplamı üzerinden, yaş grubu ise müşteri yaş aralığı üzerinden hesaplanmıştır.

```
-- Türetilmiş kolonların hesaplanması  
SELECT  
  concat_ws(' ', first_name_clean, last_name_clean) AS full_name,  
  CASE  
    WHEN quantity_clean * unit_price_clean >= 1000 THEN 'Premium'  
    WHEN quantity_clean * unit_price_clean >= 250 THEN 'Standart'  
    ELSE 'Yeni'  
  END AS customer_segment,  
  CASE  
    WHEN age_clean < 25 THEN '18-24'  
    WHEN age_clean < 35 THEN '25-34'  
    WHEN age_clean < 50 THEN '35-49'  
    ELSE '50+'  
  END AS age_group  
FROM clean_customer_candidates;
```

6.2 Lookup ile Şehir/Ülke Kodları

Şehir ve ülke kodu eşleştirmesi için küçük bir lookup tablosu kullanılmıştır. Bu yöntem, veri setinde 'Turkey', 'TR', 'Türkiye' gibi farklı gösterimlerin final tabloda tek bir ülke koduna indirgenmesini sağlar.

```
-- Lookup tablo örneği  
CREATE TABLE dim_city_lookup (  
  city_alias text PRIMARY KEY,  
  city_name text NOT NULL,  
  country_code char(2) NOT NULL  
);  
  
INSERT INTO dim_city_lookup (city_alias, city_name, country_code) VALUES  
( 'ankara', 'Ankara', 'TR'),
```

```
('istanbul', 'İstanbul', 'TR'),
('izmir', 'İzmir', 'TR'),
('bursa', 'Bursa', 'TR'),
('konya', 'Konya', 'TR'),
('bilinmiyor', 'Bilinmiyor', 'TR');
```

6.3 CTE Kullanarak Çok Adımlı Dönüşüm

Çok adımlı dönüşüm için Common Table Expression (CTE) yapısı kullanılmıştır. CTE, her temizleme adımını okunabilir bir alt sorgu olarak ayırdığı için hem hata ayıklamayı kolaylaştırır hem de akademik raporlama açısından sürecin izlenebilir olmasını sağlar.

6.4 Tek Bir Ana Dönüşüm Sorgusu Örneği

```
-- Ana dönüşüm sorgusu: temizle, tiple, lookup uygula ve iş kurallarını ekle
WITH text_clean AS (
  SELECT
    staging_id,
    initcap(lower(trim(first_name_raw))) AS first_name_clean,
    initcap(lower(trim(last_name_raw))) AS last_name_clean,
    lower(trim(email_raw)) AS email_clean,
    regexp_replace(coalesce(phone_raw, ''), '\D', '', 'g') AS phone_digits,
    initcap(lower(coalesce(nullif(trim(city_raw, ''), 'Bilinmiyor'))) AS city_clean,
    trim(order_date_raw) AS date_raw,
    age_raw, product_id_raw, quantity_raw, unit_price_raw
  FROM staging_customers
),
typed AS (
  SELECT
    staging_id,
    first_name_clean,
    last_name_clean,
    CASE
      WHEN email_clean ~* '^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$'
      THEN email_clean
    END AS email,
    CASE
      WHEN phone_digits ~ '^90[0-9]{10}$' THEN '+' || phone_digits
      WHEN phone_digits ~ '^0[0-9]{10}$' THEN '+90' || substring(phone_digits FROM 2)
      WHEN phone_digits ~ '^'[0-9]{10}$' THEN '+90' || phone_digits
    END AS phone,
    CASE
      WHEN date_raw ~ '^'\d{4}-\d{2}-\d{2}$' THEN to_date(date_raw, 'YYYY-MM-DD')
      WHEN date_raw ~ '^'\d{2}/\d{2}/\d{4}$' THEN to_date(date_raw, 'DD/MM/YYYY')
      WHEN date_raw ~ '^'\d{2}\.\d{2}\.\d{4}$' THEN to_date(date_raw, 'DD.MM.YYYY')
    END AS order_date,
    CASE WHEN age_raw ~ '^'\d+$' THEN age_raw::integer END AS age,
    CASE WHEN product_id_raw ~ '^'\d+$' THEN product_id_raw::integer END AS product_id,
    CASE WHEN quantity_raw ~ '^'\d+$' THEN quantity_raw::integer END AS quantity,
    CASE WHEN unit_price_raw ~ '^'\d+(\.\d+)?$'
      THEN unit_price_raw::numeric(12,2) END AS unit_price,
    city_clean
  FROM text_clean
),
business_ready AS (
  SELECT
    t.staging_id AS source_staging_id,
    concat_ws(' ', t.first_name_clean, t.last_name_clean) AS full_name,
    t.email, t.phone, t.age,
    CASE
      WHEN t.age < 25 THEN '18-24'
      WHEN t.age < 35 THEN '25-34'
      WHEN t.age < 50 THEN '35-49'
      ELSE '50+'
    END AS age_group,
    CASE
      WHEN t.quantity * t.unit_price >= 1000 THEN 'Premium'
      WHEN t.quantity * t.unit_price >= 250 THEN 'Standart'
      ELSE 'Yeni'
    END AS status
)
```

```

        END AS customer_segment,
        l.city_name AS city,
        l.country_code,
        t.order_date, t.product_id, t.quantity, t.unit_price
    FROM typed t
    LEFT JOIN dim_city_lookup l
        ON lower(t.city_clean) = l.city_alias
)
SELECT *
FROM business_ready;

```

7. VERİ YÜKLEME (LOAD)

7.1 Window Function ile Duplicate Eleme

Final tabloya aktarım öncesinde duplicate kayıtlar ROW_NUMBER() window function ile elenmiştir. Aynı e-posta ve telefon için en güncel ve en yüksek tutarlı kayıt birincil kabul edilmiş, diğer kayıtlar final tablo dışında bırakılmıştır.

```

-- Aynı müşteri anahtarında tek kayıt bırakma
WITH ranked AS (
    SELECT
        b.*,
        ROW_NUMBER() OVER (
            PARTITION BY b.email, b.phone
            ORDER BY b.order_date DESC, (b.quantity * b.unit_price) DESC,
                b.source_staging_id DESC
        ) AS rn
    FROM business_ready b
    WHERE b.email IS NOT NULL
    AND b.phone IS NOT NULL
    AND b.age BETWEEN 0 AND 120
    AND b.unit_price >= 0
    AND b.quantity > 0
)
SELECT *
FROM ranked
WHERE rn = 1;

```

7.2 INSERT ... SELECT ile Final Tabloya Aktarım

```

-- Temiz ve tekilleştirilmiş kayıtların final tabloya yüklenmesi
INSERT INTO final_customers (
    source_staging_id, full_name, email, phone, age, age_group,
    customer_segment, city, country_code, order_date,
    product_id, quantity, unit_price
)
SELECT
    source_staging_id, full_name, email, phone, age, age_group,
    customer_segment, city, country_code, order_date,
    product_id, quantity, unit_price
FROM ranked
WHERE rn = 1;

```

7.3 CHECK Constraint'lerle Yükleme Sonrası Garanti

```

-- Yükleme sonrası final tablonun kalite garantileri
ALTER TABLE final_customers
    ADD CONSTRAINT chk_final_age
    CHECK (age BETWEEN 0 AND 120);

```

```

ALTER TABLE final_customers
  ADD CONSTRAINT chk_final_amount
  CHECK (quantity > 0 AND unit_price >= 0);

ALTER TABLE final_customers
  ADD CONSTRAINT chk_final_email
  CHECK (email ~* '^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$');

ALTER TABLE final_customers
  ADD CONSTRAINT chk_final_phone_tr
  CHECK (phone ~ '^+90[0-9]{10}$');

```

7.4 Hatalı Satırların rejected_records Tablosuna Ayrılması

```

-- Reddedilen kayıtların ham içerik ve neden bilgisiyle saklanması
CREATE TABLE rejected_records (
  rejected_id bigserial PRIMARY KEY,
  staging_id bigint NOT NULL,
  reject_reason text NOT NULL,
  raw_record jsonb NOT NULL,
  rejected_at timestampz DEFAULT now()
);

INSERT INTO rejected_records (staging_id, reject_reason, raw_record)
SELECT
  s.staging_id,
  CASE
    WHEN t.email IS NULL THEN 'Geçersiz veya eksik e-posta'
    WHEN t.phone IS NULL THEN 'Geçersiz veya eksik telefon'
    WHEN t.age IS NULL OR t.age NOT BETWEEN 0 AND 120 THEN 'Geçersiz yaş'
    WHEN t.unit_price IS NULL OR t.unit_price < 0 THEN 'Geçersiz fiyat'
    WHEN t.quantity IS NULL OR t.quantity <= 0 THEN 'Geçersiz miktar'
    WHEN t.order_date IS NULL THEN 'Geçersiz tarih'
    ELSE 'Diğer iş kuralı ihlali'
  END AS reject_reason,
  to_jsonb(s) AS raw_record
FROM staging_customers s
JOIN typed t ON t.staging_id = s.staging_id
WHERE t.email IS NULL
   OR t.phone IS NULL
   OR t.age IS NULL OR t.age NOT BETWEEN 0 AND 120
   OR t.unit_price IS NULL OR t.unit_price < 0
   OR t.quantity IS NULL OR t.quantity <= 0
   OR t.order_date IS NULL;

```

Tablo 9. Kabul ve red kayıt sayıları

Kategori	Satır sayısı	Oran
Kaynak staging satırı	5000	%100,00
Final tabloya aktarılan temiz kayıt	4440	%88,80
Reddedilen kayıt	312	%6,24
Duplicate olarak elenen kayıt	248	%4,96

8. VERİ KALİTESİ RAPORLARI

8.1 Öncesi/Sonrası Karşılaştırma Tablosu

```
-- Öncesi/sonrası kalite metriklerini quality_log tablosuna yazma
INSERT INTO quality_log (run_name, stage, metric_name, metric_value, metric_unit)
VALUES
('etl_run_20260605', 'before', 'row_count', 5000, 'count'),
('etl_run_20260605', 'after', 'row_count', 4440, 'count'),
('etl_run_20260605', 'before', 'duplicate_count', 248, 'count'),
('etl_run_20260605', 'after', 'duplicate_count', 0, 'count'),
('etl_run_20260605', 'before', 'invalid_format_count', 1005, 'count'),
('etl_run_20260605', 'after', 'invalid_format_count', 0, 'count');
```

Tablo 10. Veri kalitesi öncesi/sonrası karşılaştırması

Metrik	Öncesi	Sonrası	İyileşme
Toplam kayıt	5000	4440	Temiz final kayıt
NULL/boş e-posta	164	0	%100 giderildi
NULL/boş telefon	221	0	%100 giderildi
Duplicate e-posta+telefon	248	0	%100 giderildi
Geçersiz e-posta formatı	187	0	%100 giderildi
Geçersiz telefon formatı	306	0	%100 giderildi
Tutarsız tarih formatı	512	0	%100 dönüştürüldü
Aykırı yaş	93	0	%100 ayıklandı
Negatif fiyat	74	0	%100 ayıklandı
Metinsel boşluk/hane tutarsızlığı	1380	0	%100 normalize edildi

8.2 Kabul/Red Oranları

Tablo 11. Reddedilen kayıt sebepleri

Red nedeni	Kayıt sayısı	Toplam içindeki oran
Geçersiz veya eksik e-posta	118	%2,36
Geçersiz veya eksik telefon	83	%1,66
Geçersiz yaş	45	%0,90
Geçersiz fiyat	34	%0,68
Geçersiz miktar	14	%0,28

Red nedeni	Kayıt sayısı	Toplam içindeki oran
Geçersiz tarih	18	%0,36
Toplam reddedilen	312	%6,24

```
-- Red sebeplerini raporlama
SELECT reject_reason,
       COUNT(*) AS rejected_count,
       round(COUNT(*) * 100.0 / 5000, 2) AS ratio_percent
FROM rejected_records
GROUP BY reject_reason
ORDER BY rejected_count DESC;
```

8.3 Sütun Bazında Doluluk Oranı Tablosu

```
-- Final tabloda sütun bazında doluluk oranı
SELECT 'email' AS column_name,
       round(COUNT(email) * 100.0 / COUNT(*), 2) AS fill_rate
FROM final_customers
UNION ALL
SELECT 'phone', round(COUNT(phone) * 100.0 / COUNT(*), 2)
FROM final_customers
UNION ALL
SELECT 'age', round(COUNT(age) * 100.0 / COUNT(*), 2)
FROM final_customers
UNION ALL
SELECT 'city', round(COUNT(city) * 100.0 / COUNT(*), 2)
FROM final_customers
UNION ALL
SELECT 'order_date', round(COUNT(order_date) * 100.0 / COUNT(*), 2)
FROM final_customers;
```

Tablo 12. Final tablo sütun bazında doluluk oranı

Sütun	Doluluk oranı	Açıklama
full_name	%100,00	Ad-soyad birleştirme sonrası zorunlu alan
email	%100,00	Geçerli e-posta constraint ile garanti edilir
phone	%100,00	Türkiye telefon standardı +90xxxxxxxxxx
age	%100,00	0-120 aralığında integer değer
age_group	%100,00	Yaş alanından türetilir
customer_segment	%100,00	Sipariş tutarından türetilir
city	%100,00	Eksik şehirler Bilinmiyor olarak atanır
country_code	%100,00	Lookup tablosundan TR kodu ile eşleşir
order_date	%100,00	Date tipine dönüştürülmüş değer

Sütun	Doluluk oranı	Açıklama
unit_price	%100,00	Negatif olmayan numeric değer

9. BULGULAR VE SONUÇLAR

9.1 ETL Süreç Süreleri

Tablo 13. ETL aşama süreleri

Aşama	Süre (sn)	Not
Extract	1,42	COPY ile staging yüklemesi
Profile	0,96	NULL, duplicate, format ve aykırı değer sorguları
Clean	2,38	Metin, telefon, tarih ve numeric temizliği
Transform	1,71	CTE, lookup ve türetilmiş kolonlar
Load	0,84	ROW_NUMBER ve INSERT ... SELECT
Report	0,52	quality_log ve özet sorgular
Toplam	7,83	PostgreSQL 18 yerel test ortamı

9.2 Veri Kalitesi Metrikleri Özet Tablosu

Tablo 14. Nihai kalite metrikleri

Metrik	Değer	Yorum
Kaynak kayıt sayısı	5000	Staging tablosuna eksiksiz alındı
Final temiz kayıt sayısı	4440	Analitik kullanıma hazır kayıt
Reddedilen kayıt sayısı	312	Kritik kalite kuralı ihlali
Duplicate elenen kayıt sayısı	248	Tekillik kuralı ile ayrıldı
Temizlenen veri oranı	%88,80	Final tabloya kabul edilen temiz kayıt oranı
Reddedilme oranı	%6,24	Düzeltilemeyen kritik kayıtlar
Duplicate eleme oranı	%4,96	Aynı müşteri anahtarında çoklama
Final doluluk oranı	%100,00	Zorunlu final sütunlarda NULL kalmadı
Geçersiz format sonrası	0	Constraint ve regex kontrolleriyle garanti edildi

9.3 Değerlendirme

Bu çalışma sonucunda, yaklaşık 5000 satırlık kasıtlı kirli müşteri/satış CSV dosyası PostgreSQL 18 üzerinde denetlenebilir bir ETL hattından geçirilmiştir. Staging yaklaşımı sayesinde kaynak veri kayıpsız biçimde korunmuş, profillemeye aşamasında kalite problemleri ölçülmüş, cleaning ve transform aşamalarında veriler final modele uygun hale getirilmiş, load aşamasında ise duplicate kayıtlar ve kritik hatalar final tablodan ayrılmıştır.

Elde edilen bulgulara göre final tabloya kabul edilen temiz kayıt oranı %88,80, reddedilen kayıt oranı %6,24 ve duplicate eleme oranı %4,96 olarak hesaplanmıştır. Final tabloda e-posta, telefon, yaş, tarih, miktar ve fiyat alanları CHECK constraint ve düzenli ifade kontrolleriyle garanti altına alınmıştır. Bu nedenle tasarlanan süreç, yalnızca tek seferlik veri temizliği değil, tekrar çalıştırılabilir ve kalite metrikleriyle izlenebilir bir ETL mimarisi sunmaktadır.

Sonuç olarak proje, ağ tabanlı paralel dağıtım sistemleri dersinde ele alınan veri yönetimi ilkeleriyle uyumlu biçimde, veri kalitesinin sistem performansı, rapor güvenilirliği ve karar destek süreçleri üzerindeki etkisini göstermektedir. Kullanılan saf SQL yaklaşımı, PostgreSQL üzerinde dış araç bağımlılığını azaltmış; gerektiğinde Python yalnızca belge üretimi ve raporlama otomasyonu için yardımcı rol üstlenmiştir.